

¿Por qué programación concurrente?

Velocidad de ejecución, aprovechando el hardware paralelo: Múltiples procesadores o múltiples *cores* en un procesador

Captura la estructura lógica del problema: Servidores, aplicaciones gráficas

Distribución física: E.g., aplicaciones que corren a través de la Internet; aplicaciones embebidas, como los sistemas de control de un auto

Programas cuya velocidad de ejecución está limitada por operaciones de I/O o el ancho de banda de la memoria

Mecanismos de sincronización controlados por el *scheduler*: Semáforos y monitores

Cuando un proceso se pone a esperar que ocurra un evento, se asocia el evento con información que identifica al proceso

→ cuando el evento ocurre, el *scheduler* sabe que el proceso correspondiente está listo para ser reanudado

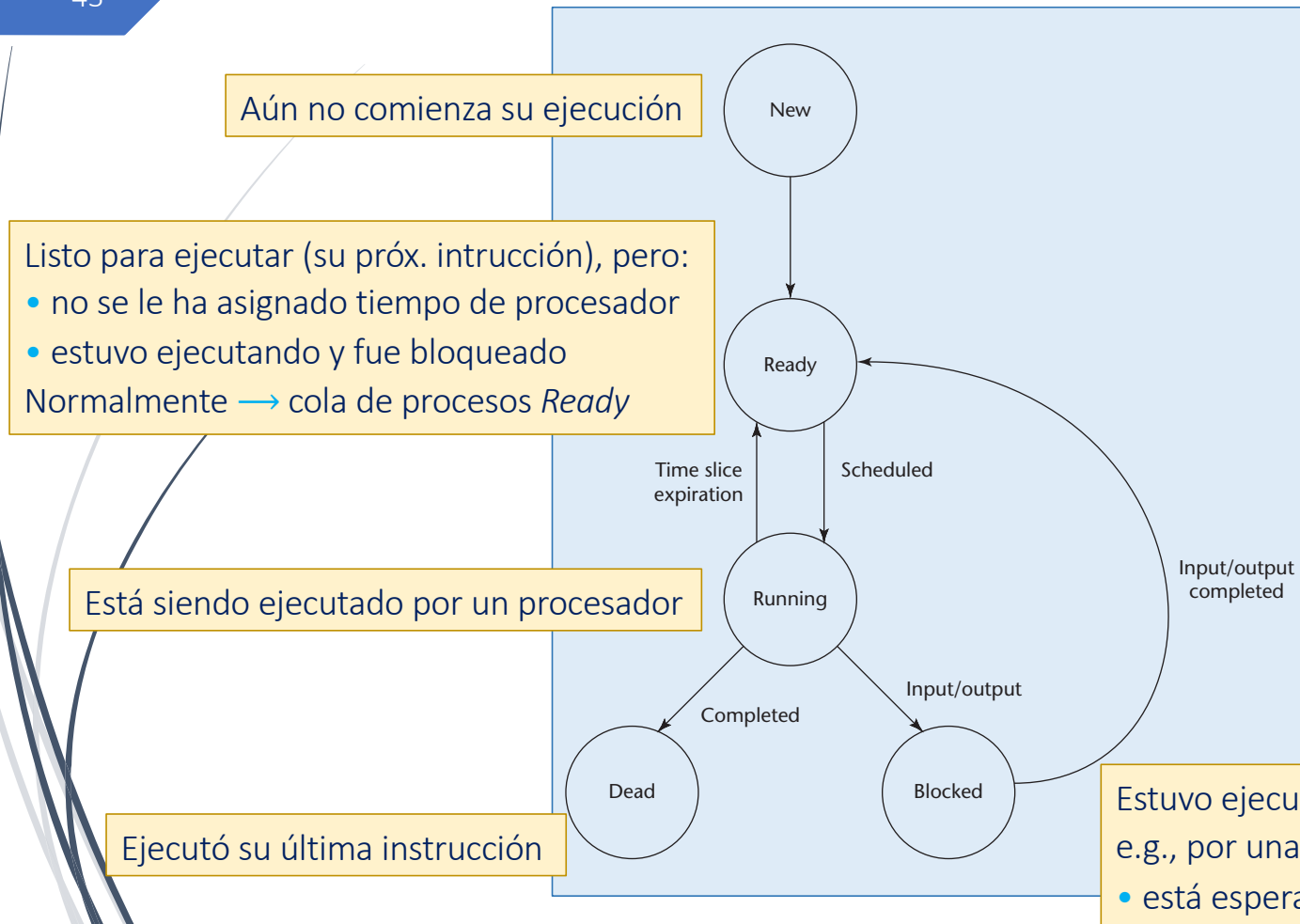
Implementados mediante variables

... que pueden tomar ciertos valores

... y a las que se les puede aplicar ciertas operaciones

→ tipos de datos

Diagrama de estados de un proceso



Semáforo

Estructura de datos:

- un entero *counter*
- una cola *Q* de descriptores de procesos
- una operación *P* (o *wait*)
- una operación *V* (o *release*)

Almacena solicitudes de acceso a la sección crítica que aún no pueden ser concedidas

```
void P(sem S){  
    if S.counter > 0 then  
        S.counter = S.counter - 1;  
    else {  
        S.Q = enqueue(S.Q, p);  
        p.state = blocked;  
    }  
}
```

```
void V(sem S){  
    if S.Q = ∅ then  
        S.counter = S.counter + 1;  
    else {  
        q = dequeue(S.Q);  
        q.state = ready;  
    }  
}
```

Si dos procesos tratar de ejecutar al mismo tiempo $P(S)$, sólo uno tendrá éxito

Múltiples productores y consumidores, *buffer* de capacidad 1

Un único semáforo garantiza exclusión mutua, pero no puede garantizar además alternancia

```
bufferType buffer
sem empty ← 1, full ← 0

process Producer[i ← 1 ... m]:
  while (true):
    produce data
    P(empty)
    buffer.deposit(data)
    V(full)

process Consumer[j ← 1 ... n]:
  while (true):
    P(full)
    result ← buffer.fetch()
    V(empty)
    consume result
```

Asociamos un semáforo **binario** con cada estado del *buffer*

Un productor y un consumidor, buffer de capacidad $n > 1$

```
bufferType[n] buffer
int front  $\leftarrow$  0, rear  $\leftarrow$  0
sem empty  $\leftarrow$  n, full  $\leftarrow$  0
```

C/u de estos semáforos es
un contador de recursos

```
process Producer —único productor
  while (true):
    produce data
    P(empty)
    buffer[rear].deposit(data)
    rear  $\leftarrow$  (rear+1)%n
    V(full)
```

```
process Consumer —único consumidor
  while (true):
    P(full)
    result  $\leftarrow$  buffer[front].fetch()
    front  $\leftarrow$  (front+1)%n
    V(empty)
    consume result
```

deposit y *fetch* pueden ejecutarse
concurrentemente si al mismo
tiempo hay un casillero vacío y un
data almacenado —tienen acceso a
casilleros diferentes del *buffer* y no
interferirán entre ellas

Múltiples productores y consumidores, buffer de capacidad $n > 1$

```
bufferType[n] buf
int front ← 0, rear ← 0
sem empty ← n, full ← 0
sem mutexD ← 1, mutexF ← 1

process Producer[i ← 1 ... m]:
  while (true):
    produce data
    P(empty)
    P(mutexD)
    buf[rear].dep(data)
    rear ← (rear+1)%n
    V(mutexD)
    V(full)
```

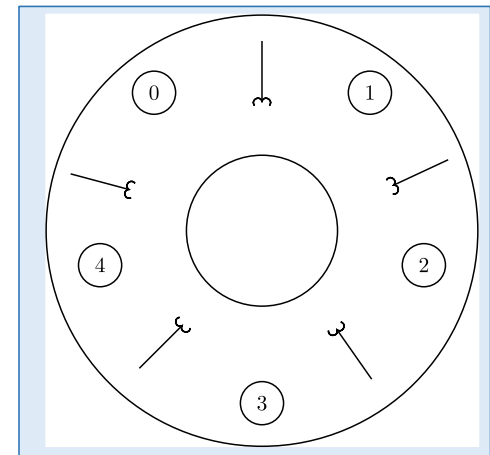
Las operaciones *deposit* deben ocurrir bajo exclusión mutua

```
process Consumer[j ← 1 ... n]:
  while (true):
    P(full)
    P(mutexF)
    result ← buf[front].fetch()
    front ← (front+1)%n
    V(mutexF)
    V(empty)
    consume result
```

Las operaciones *fetch* deben ocurrir bajo exclusión mutua

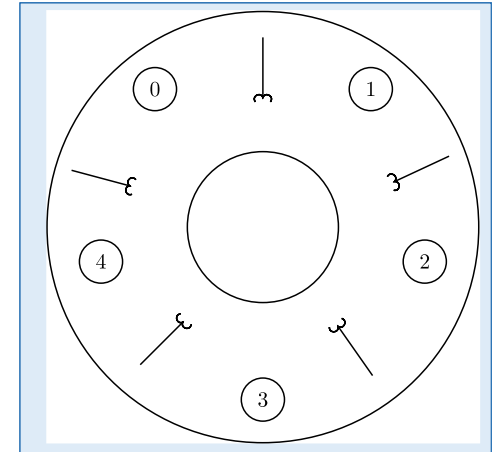
El problema de los filósofos comensales

Cinco filósofos viven pensando y comiendo, alternadamente
Para comer, se sientan alrededor de una mesa circular, que en el centro tiene un gran plato de tallarines, y en la que hay 5 tenedores dispuestos cada uno entre dos puestos de filósofos
Un filósofo debe obtener los dos tenedores —a su derecha y a su izquierda— para servirse los tallarines (y poder comer)



El problema de los filósofos comensales

Cinco filósofos viven pensando y comiendo, alternadamente. Para comer, se sientan alrededor de una mesa circular, que en el centro tiene un gran plato de tallarines, y en la que hay 5 tenedores dispuestos cada uno entre dos puestos de filósofos. Un filósofo debe obtener los dos tenedores —a su derecha y a su izquierda— para servirse los tallarines (y poder comer).



Escribir un programa que simule el comportamiento de los filósofos ... evitando la situación en la que todos los filósofos tienen hambre pero ninguno puede obtener los dos tenedores que necesita.

Es claro que:

- dos filósofos vecinos no pueden comer al mismo tiempo
- a lo más dos filósofos pueden comer al mismo tiempo

Los tenedores son el recurso compartido
Simulamos las acciones de los filósofos así:

```
process Phil[i]:  
  while (true):  
    pensar  
    obtener tenedores  
    comer  
    soltar tenedores
```

Hay que programar las acciones *obtener tenedores* y *soltar tenedores*

Cada tenedor es como un *lock* de una sección crítica

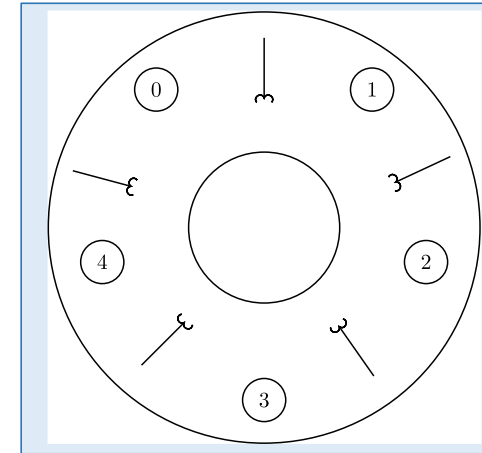
→ a lo más lo puede tener un filósofo a la vez

Representamos los tenedores por un arreglo de semáforos binarios, inicializados en 1

→ obtener un tenedor consiste en ejecutar una operación *P* sobre el semáforo correspondiente

→ soltar un tenedor consiste en ejecutar una operación *V*

El problema de los filósofos comensales: Primer intento de solución



Los tenedores son el recurso compartido
Simulamos las acciones de los filósofos así:

```
process Phil[i]:  
  while (true):  
    pensar  
    obtener tenedores  
    comer  
    soltar tenedores
```

Hay que programar las acciones *obtener tenedores* y *soltar tenedores*

Cada tenedor es como un *lock* de una sección crítica

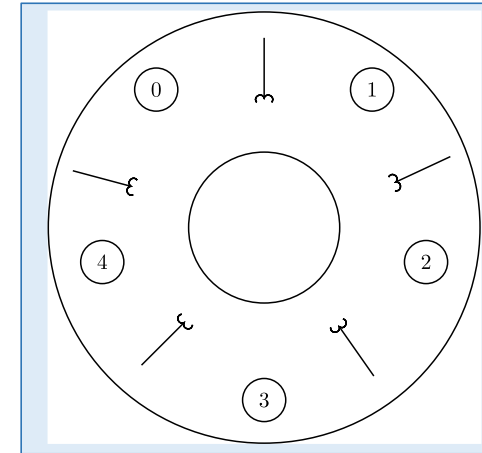
→ a lo más lo puede tener un filósofo a la vez

Representamos los tenedores por un arreglo de semáforos binarios, inicializados en 1

→ obtener un tenedor consiste en ejecutar una operación *P* sobre el semáforo correspondiente

→ soltar un tenedor consiste en ejecutar una operación *V*

El problema de los filósofos comensales: Primer intento de solución



```
sem[] fork ← {1,1,1,1,1}
```

```
process Phil[i]:  
  while (true):  
    pensar  
    P(fork[i]); P(fork[i+1])  
    comer  
    V(fork[i]); V(fork[i+1])
```

```
sem[] fork ← {1,1,1,1,1}
```

```
process Phil[i]:  
  while (true):  
    pensar  
    P(fork[i]); P(fork[i+1])  
    comer  
    V(fork[i]); V(fork[i+1])
```

El problema de los filósofos comensales: Solución

Puede producir **deadlock**:

- p.ej., si todos los filósofos han tomado sus tenedores izquierdos, entonces esperarán para siempre tratando de tomar sus tenedores derechos

Una condición necesaria para *deadlock* es que haya *espera circular*

→ un proceso espera por un recurso en poder de un segundo proceso
... éste espera por un recurso en poder de un tercero
... y así hasta un proceso que espera por un recurso en poder del primer proceso

El problema de los filósofos comensales: Solución

```
sem[] fork ← {1,1,1,1,1}
```

```
process Phil[i]:  
  while (true):  
    pensar  
    P(fork[i]); P(fork[i+1])  
    comer  
    V(fork[i]); V(fork[i+1])
```

```
sem[] fork ← {1,1,1,1,1}
```

```
process Phil[i ← 1,2,3, 4]:  
  while (true):  
    pensar  
    P(fork[i]); P(fork[i+1])  
    comer  
    V(fork[i]); V(fork[i+1])
```

```
process Phil[5]:  
  while (true):  
    pensar  
    P(fork[1]); P(fork[5])  
    comer  
    V(fork[1]); V(fork[5])
```

Puede producir **deadlock**:

- p.ej., si todos los filósofos han tomado sus tenedores izquierdos, entonces esperarán para siempre tratando de tomar sus tenedores derechos

Una condición necesaria para *deadlock* es que haya *espera circular*

→ un proceso espera por un recurso en poder de un segundo proceso
... éste espera por un recurso en poder de un tercero
... y así hasta un proceso que espera por un recurso en poder del primer proceso

Aseguremos que no ocurra *espera circular*:

→ un filósofo toma primero su tenedor derecho